Portail SSO adossé à un annuaire

Mini-projet IAM n°1 — Polycorp : OpenLDAP, Keycloak, SAML 2.0 et OpenID Connect

LDAP | SAML 2.0 | OAuth 2.0 | OpenID Connect | Keycloak 26.8 | Docker Compose

Réalisé par

Ghassen Cheyah & Anas Abaza

Novembre 2026

Sommaire

1. Contexte et objectifs
2. Architecture de la solution
3. Conception de l'annuaire : DIT, LDIF et ACL
4. Connexion de Keycloak à l'annuaire
5. Correspondance attributs, groupes, claims et rôles
6. Les deux portails et le contrôle d'accès RH
7. Démonstrations
    1. Un seul mot de passe pour deux portails
    2. Assertion SAML et ID token
    3. Emma authentifiée mais refusée
    4. Propagation des changements LDAP
    5. Le mot de passe n'est que dans l'annuaire
8. Kerberos : intégration possible avec la fédération LDAP
9. Sécurité, limites et recommandations
10. Conclusion
11. Annexe : procédure de reproduction

1. Contexte et objectifs

Polycorp dispose déjà d'un annuaire **OpenLDAP** qui contient ses comptes et ses groupes. L'entreprise veut ouvrir deux applications web sans recréer ces comptes : un **Portail RH** qui parle **SAML 2.0** et un **Portail Notes** qui parle **OpenID Connect** (OIDC, construit sur OAuth 2.0). Un utilisateur ne doit saisir son mot de passe qu'une seule fois pour accéder aux deux applications (*Single Sign-On*).

Nous avons placé **Keycloak** entre l'annuaire et les applications. Il joue à la fois le rôle de fournisseur d'identité (IdP) SAML et de serveur d'autorisation OIDC. Il ne stocke pas les comptes : il les lit dans LDAP au travers d'une *fédération*, avec un compte de service qui n'a que des droits de lecture.

Travail demandé

Exigence	Réalisation	Section
Concevoir le DIT et les LDIF (unités, utilisateurs, groupes rh et employes)	3 OU, 3 utilisateurs, 2 groupes, 1 compte de service, ACL dédiées	3
Connecter Keycloak à l'annuaire avec un compte de service limité, en lecture seule	Fédération LDAP `READ_ONLY`, bind `cn=svc-keycloak`, ACL testées	4
Faire correspondre attributs et groupes LDAP aux attributs SAML, claims OIDC et rôles	Mappers LDAP, groupes → rôles, mappers SAML et OIDC	5
Reprendre les portails du TP4 et réserver l'espace RH au groupe rh	SP SAML (8181), client OIDC (8182), API (8183), contrôle `GROUPE_RH`	6

Démonstrations attendues

Démonstration	Résultat obtenu
Alice ouvre les deux portails en ne saisissant son mot de passe qu'une fois	✓ Une session Keycloak, deux clients (section 7.1)
Le même utilisateur apparaît dans l'assertion SAML et dans l'ID token	✓ Comparaison champ par champ (section 7.2)
Emma est authentifiée mais refusée sur l'espace RH	✓ HTTP 403 sur /rh et sur POST /notes
Un changement de mot de passe ou de groupe dans LDAP se répercute : mesurer quand	✓ 0,26 s à 0,32 s sans cache, 51,97 s avec cache
Le mot de passe n'est stocké que dans l'annuaire	✓ Credential « Provided by ldap », hash visible uniquement par l'admin LDAP

2. Architecture de la solution

Toute la plateforme tourne dans **Docker Compose** (projet `tp4`). Les ports ne sont publiés que sur `127.0.0.1`. Le navigateur joint Keycloak sur `localhost:8180` ; les conteneurs le joignent en interne sur `keycloak:8080`.

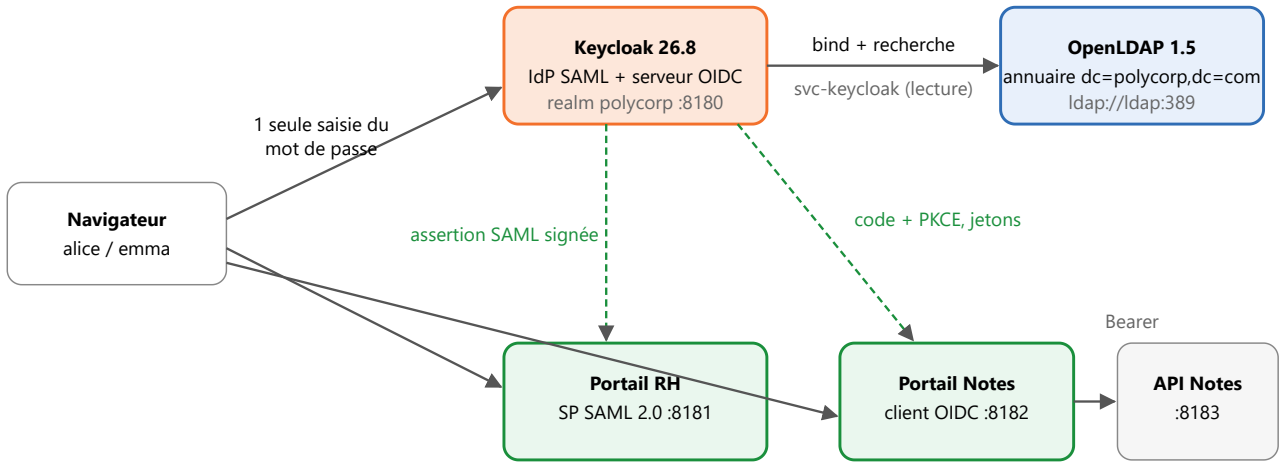


Figure 1 – *Architecture : l'annuaire reste la seule source des comptes, Keycloak fédère, les portails font confiance à Keycloak*

Service	Image / technologie	Port	Rôle
ldap	osixia/openldap 1.5.0	127.0.0.1:1389	Annuaire : comptes, groupes, mots de passe (hash SSHA)
keycloak	quay.io/keycloak/keycloak 26.8.1	127.0.0.1:8180	IdP SAML et serveur d'autorisation OIDC, realm `polycorp`
sp-saml	Flask + python3-saml	127.0.0.1:8181	Portail RH, fournisseur de service SAML
client-oauth	Flask + PyJWT	127.0.0.1:8182	Portail Notes, client OIDC public (code + PKCE S256)
api-notes	Flask + PyJWT	127.0.0.1:8183	Serveur de ressources : vérifie signature, iss, aud, exp, scope et rôle

> **Remarque de déploiement.** Les trois applications partagent la même image `tp4-apps:1.0`. À l'origine, les trois services déclaraient chacun `build:` avec ce même tag, et BuildKit échouait avec *image already exists*. Seul `sp-saml` construit désormais l'image ; `client-oauth` et `api-notes` la réutilisent.

3. Conception de l'annuaire : DIT, LDIF et ACL

3.1 Arbre de l'annuaire (DIT)

Nous avons séparé les personnes, les groupes et les comptes techniques dans trois unités organisationnelles. Ce découpage permet d'écrire des ACL simples : le compte de service voit `ou=people` et `ou=groups`, mais pas `ou=services`.

Structure du DIT

```
dc=polycorp,dc=com
├── ou=people                      (organizationalUnit)
│   ├── uid=alice   Alice Martin   (inetOrgPerson, userPassword {SSHA})
│   ├── uid=bob     Bob Durand     (inetOrgPerson)
│   └── uid=emma    Emma Petit     (inetOrgPerson)
├── ou=groups                      (organizationalUnit)
│   ├── cn=rh        member: alice                (groupOfNames)
│   └── cn=employes  member: alice, bob, emma     (groupOfNames)
└── ou=services                    (organizationalUnit)
    └── cn=svc-keycloak            (organizationalRole + simpleSecurityObject)
```

Élément	Choix	Justification
Utilisateurs	`inetOrgPerson`, RDN `uid`	Classe standard : `uid`, `cn`, `givenName`, `sn`, `mail`, `userPassword`
Groupes	`groupOfNames`, attribut `member` (DN)	Appartenance lisible et directement exploitable par le mapper de groupes de Keycloak
Mots de passe	`{SSHA}`	Jamais en clair dans l'annuaire : SHA-1 salé
Compte de service	`organizationalRole` + `simpleSecurityObject`	Compte technique sans boîte mail ni identité de personne, seulement un mot de passe

3.2 Fichier LDIF chargé au démarrage

TP4/ldap/ldif/01-structure.ldif

```
dn: ou=people,dc=polycorp,dc=com
objectClass: organizationalUnit
ou: people

dn: ou=groups,dc=polycorp,dc=com
objectClass: organizationalUnit
ou: groups

dn: ou=services,dc=polycorp,dc=com
objectClass: organizationalUnit
ou: services

dn: uid=alice,ou=people,dc=polycorp,dc=com
objectClass: inetOrgPerson
uid: alice
cn: Alice Martin
givenName: Alice
sn: Martin
mail: alice.martin@polycorp.test
userPassword: {SSHA}SrkoK5J1J9iYsvnYFESPeJdx5eZ+S0xSVFZVbw==

dn: uid=bob,ou=people,dc=polycorp,dc=com
objectClass: inetOrgPerson
uid: bob
cn: Bob Durand
givenName: Bob
sn: Durand
mail: bob.durand@polycorp.test
userPassword: {SSHA}SrkoK5J1J9iYsvnYFESPeJdx5eZ+S0xSVFZVbw==

dn: uid=emma,ou=people,dc=polycorp,dc=com
objectClass: inetOrgPerson
uid: emma
cn: Emma Petit
givenName: Emma
sn: Petit
mail: emma.petit@polycorp.test
userPassword: {SSHA}SrkoK5J1J9iYsvnYFESPeJdx5eZ+S0xSVFZVbw==

dn: cn=svc-keycloak,ou=services,dc=polycorp,dc=com
objectClass: organizationalRole
objectClass: simpleSecurityObject
cn: svc-keycloak
userPassword: {SSHA}/I1T1fhhv7FynPrTOzEidmsvjiqpBX6pxhkOzw==

dn: cn=rh,ou=groups,dc=polycorp,dc=com
objectClass: groupOfNames
cn: rh
member: uid=alice,ou=people,dc=polycorp,dc=com

dn: cn=employes,ou=groups,dc=polycorp,dc=com
objectClass: groupOfNames
cn: employes
member: uid=alice,ou=people,dc=polycorp,dc=com
member: uid=bob,ou=people,dc=polycorp,dc=com
member: uid=emma,ou=people,dc=polycorp,dc=com
```

3.3 ACL : un compte de service à droits limités

Les ACL sont appliquées sur la base `mdb` par `ldapmodify -Y EXTERNAL -H ldapi:///` : l'authentification se fait par le socket Unix, en tant que root du conteneur, sans mot de passe. Les règles sont évaluées dans l'ordre et la première qui correspond s'applique.

TP4/ldap/02-acl.ldif

```
dn: olcDatabase={1}mdb,cn=config
changetype: modify
replace: olcAccess
olcAccess: {0}to *
  by dn.exact="gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth" manage
  by * break
olcAccess: {1}to attrs=userPassword
  by anonymous auth
  by * none
olcAccess: {2}to dn.subtree="ou=people,dc=polycorp,dc=com"
  by dn.exact="cn=svc-keycloak,ou=services,dc=polycorp,dc=com" read
  by * none
olcAccess: {3}to dn.subtree="ou=groups,dc=polycorp,dc=com"
  by dn.exact="cn=svc-keycloak,ou=services,dc=polycorp,dc=com" read
  by * none
olcAccess: {4}to *
  by * none
```

Règle	Effet
{0}	Root local (socket `ldapi`) : `manage` sur tout, sinon on passe aux règles suivantes (`break`)
{1}	`userPassword` : utilisable seulement pour s'authentifier (`anonymous auth`), illisible pour tous les autres
{2} / {3}	`svc-keycloak` peut lire `ou=people` et `ou=groups`, et rien d'autre
{4}	Refus par défaut de tout le reste

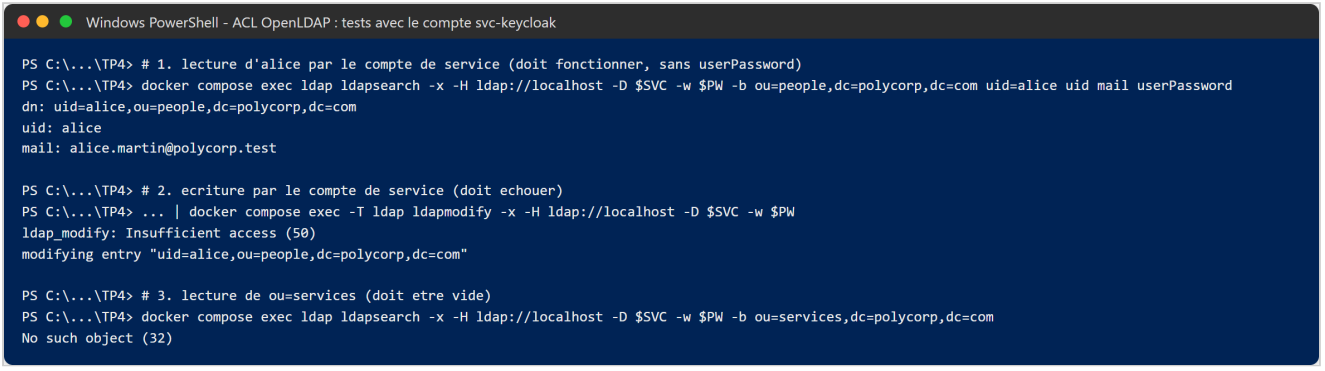


Figure 2 – *Tests des ACL : lecture autorisée sans le mot de passe, écriture refusée (50), ou=services invisible (32)*

4. Connexion de Keycloak à l'annuaire

Dans le realm `polycorp`, nous avons créé un fournisseur **User federation** → **LDAP**. Keycloak s'y connecte avec le compte `cn=svc-keycloak,ou=services,dc=polycorp,dc=com`, en **lecture seule**.

Paramètre	Valeur	Pourquoi
Connection URL	`ldap://ldap:389`	Nom du service dans le réseau Docker (LDAPS en production)
Bind DN	`cn=svc-keycloak,ou=services,…`	Compte dédié, jamais l'administrateur de l'annuaire
Edit mode	`READ_ONLY`	Keycloak ne modifie jamais l'annuaire, qui reste la source de vérité
Users DN / filtre	`ou=people,dc=polycorp,dc=com`, `inetOrgPerson`, scope un niveau	Seules les personnes sont des comptes utilisateurs
Username / RDN / UUID	`uid` / `uid` / `entryUUID`	Identifiant stable côté LDAP, même si un attribut change
Import users	On	Keycloak garde une copie locale des fiches (sans mot de passe). Le `sub` est un UUID Keycloak
Cache policy	`NO_CACHE` (testé aussi en `MAX_LIFESPAN` 1 min)	Voir la mesure de propagation (section 7.4)

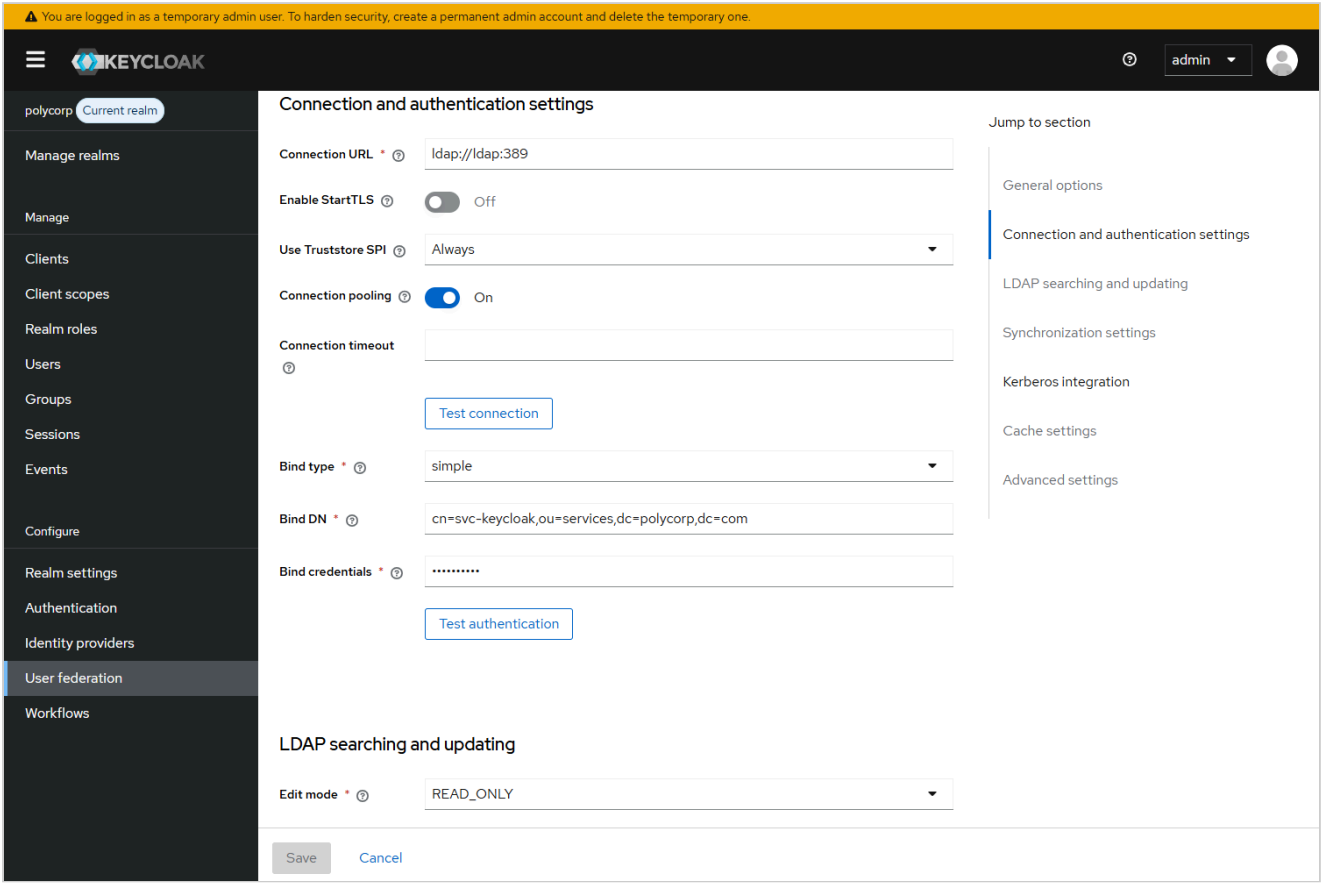


Figure 3 – *Connexion et authentification : URL, bind simple avec le compte de service, mode READ_ONLY*

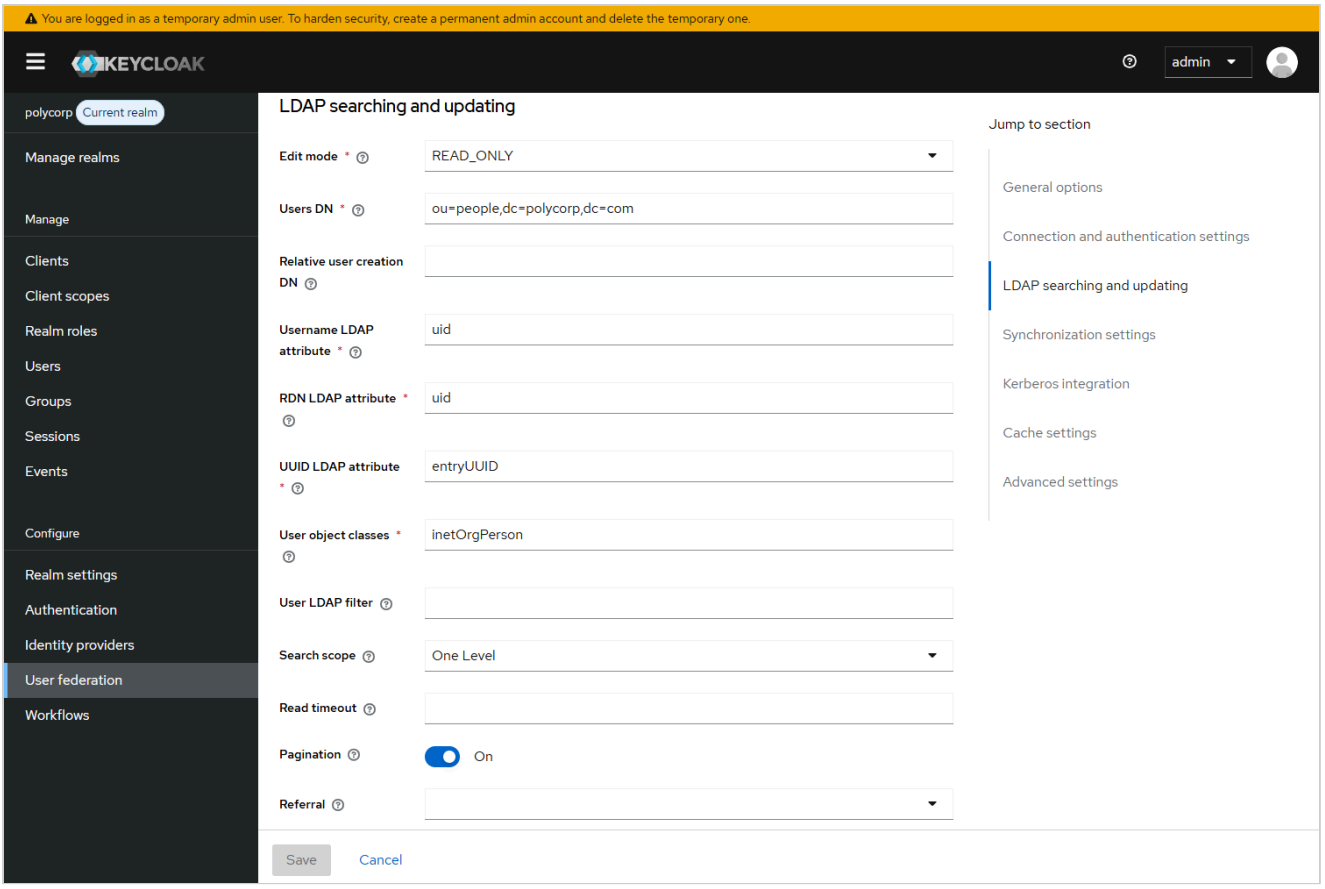


Figure 4 – *Recherche LDAP : DN des utilisateurs, attributs uid / entryUUID, classe inetOrgPerson*

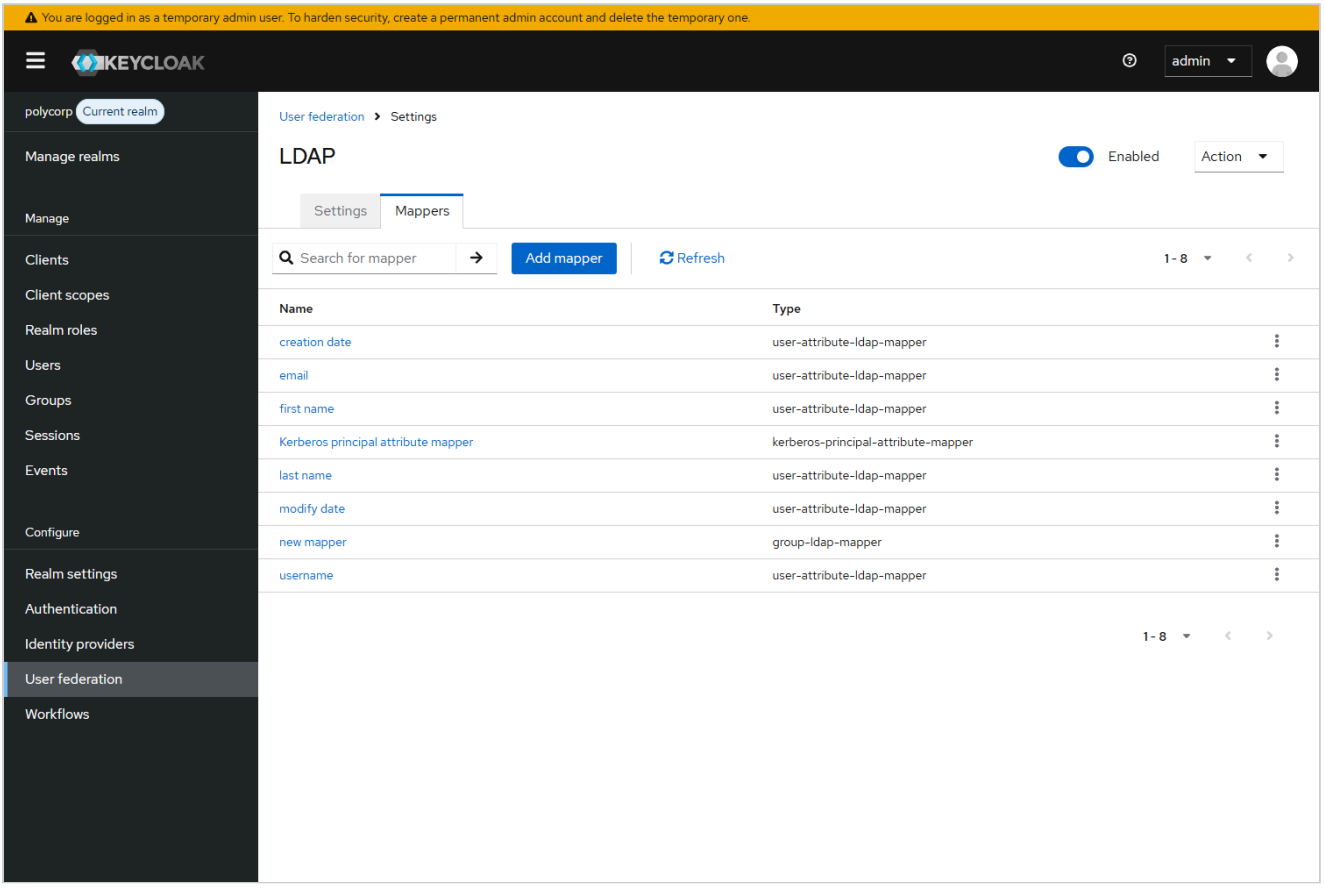


Figure 5 – *Mappers de la fédération : attributs utilisateur et mapper de groupes*

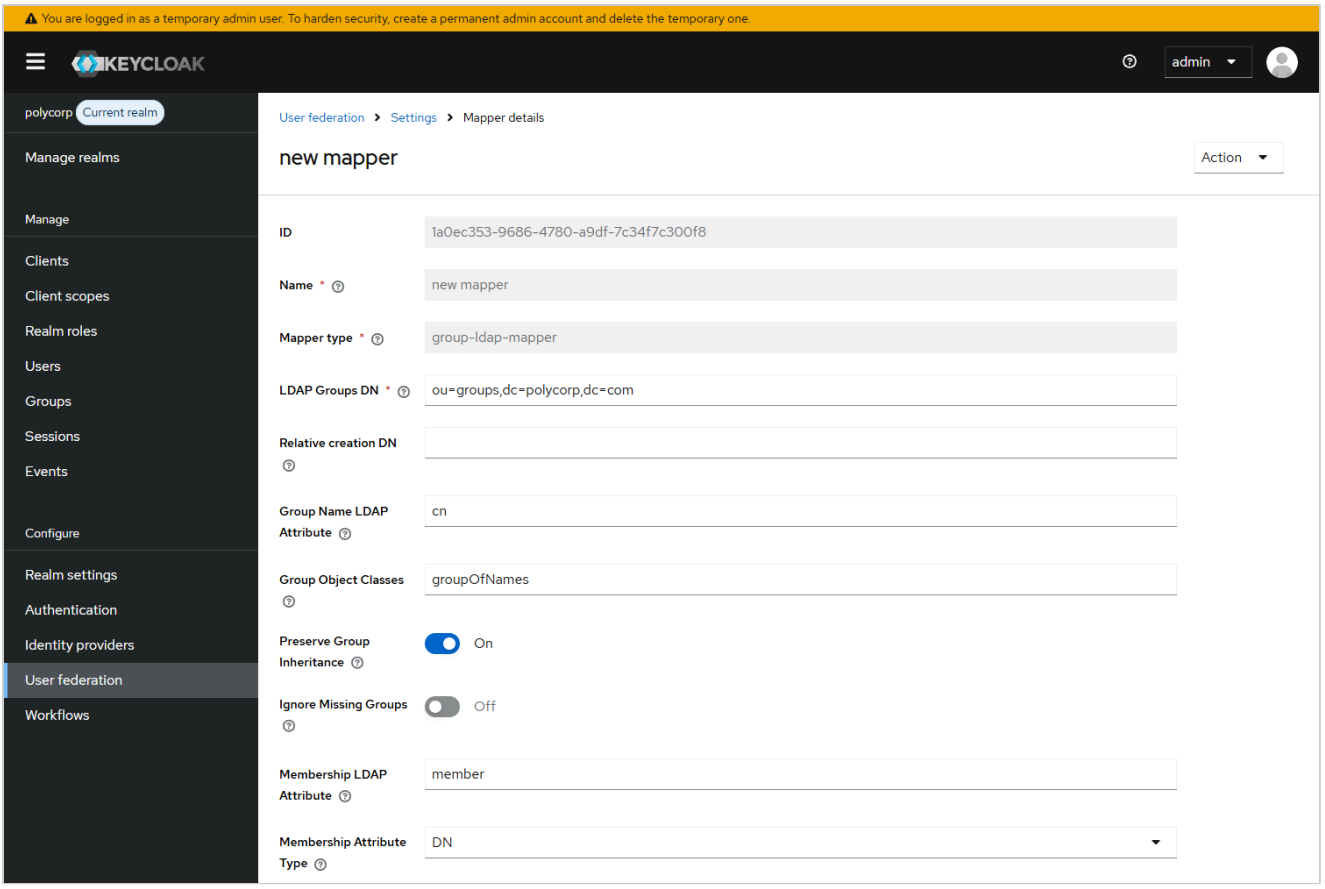


Figure 6 – *Mapper de groupes : groupOfNames sous ou=groups, appartenance par l'attribut member (DN), mode READ_ONLY*

Correction apportée. Le mapper « first name » créé par défaut lisait `cn` (« Alice Martin »). Le jeton affichait donc `"name": "Alice Martin Martin"`. Nous l'avons fait pointer vers `givenName`, puis lancé une synchronisation complète : « 0 imported users, 3 updated users ». L'ID token contient désormais `given_name: Alice`, `family_name: Martin`.

5. Correspondance attributs, groupes, claims et rôles

Une même information LDAP traverse trois niveaux : la fédération la convertit en attribut ou en groupe Keycloak, les groupes portent des rôles de realm, puis chaque client publie ce dont il a besoin dans son protocole.

LDAP	Keycloak	Assertion SAML (Portail RH)	Jetons OIDC (Portail Notes)
`uid`	username	`NameID` (format username)	`preferred_username`
`entryUUID`	ID LDAP de l'utilisateur ; ID Keycloak = UUID	–	`sub` (UUID Keycloak)
`mail`	email	attribut `email`	`email`
`givenName` / `sn`	firstName / lastName	–	`given_name`, `family_name`, `name`
`cn=rh` (member)	groupe `rh` → rôles `rh`, `editeur`	attributs `groups` et `Role`	`groups`, `realm_access.roles`
`cn=employes` (member)	groupe `employes` → rôle `employe`	attributs `groups` et `Role`	`groups`, `realm_access.roles`

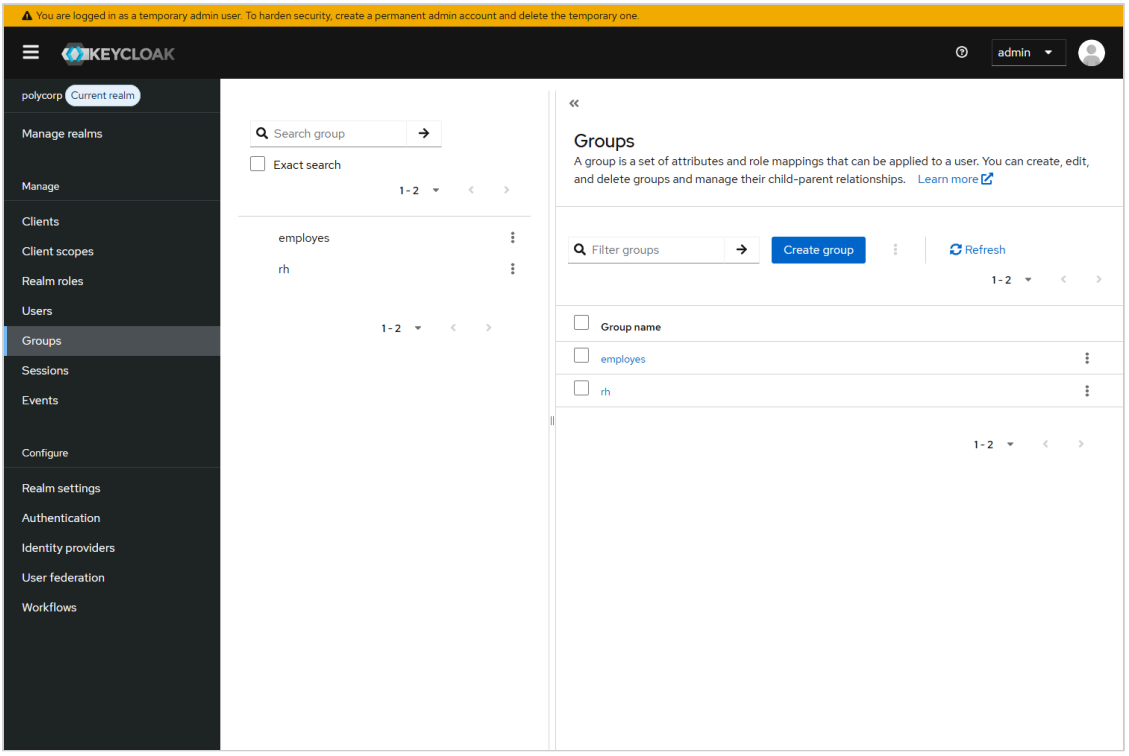


Figure 7 – *Groupes importés depuis LDAP : employes et rh*

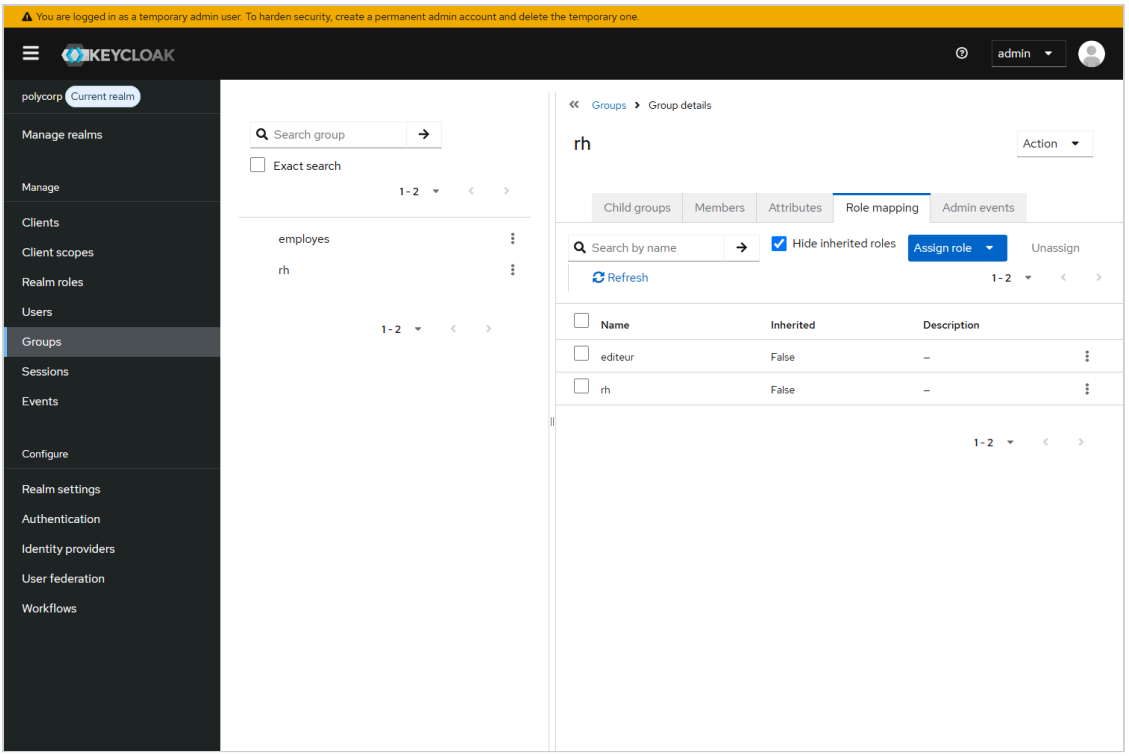


Figure 8 – *Le groupe rh porte les rôles de realm rh et editeur*

Côté SAML

Le client SAML `http://localhost:8181/saml/metadata` signe la réponse et l'assertion (RSA-SHA256), force le binding POST vers l'ACS `/saml/acs` et utilise le format de NameID *username*. Le scope par défaut `role_list` émet l'attribut `Role`. Nous avons ajouté un mapper **Group list** (`groups`, sans chemin complet) pour transmettre les groupes LDAP tels quels.

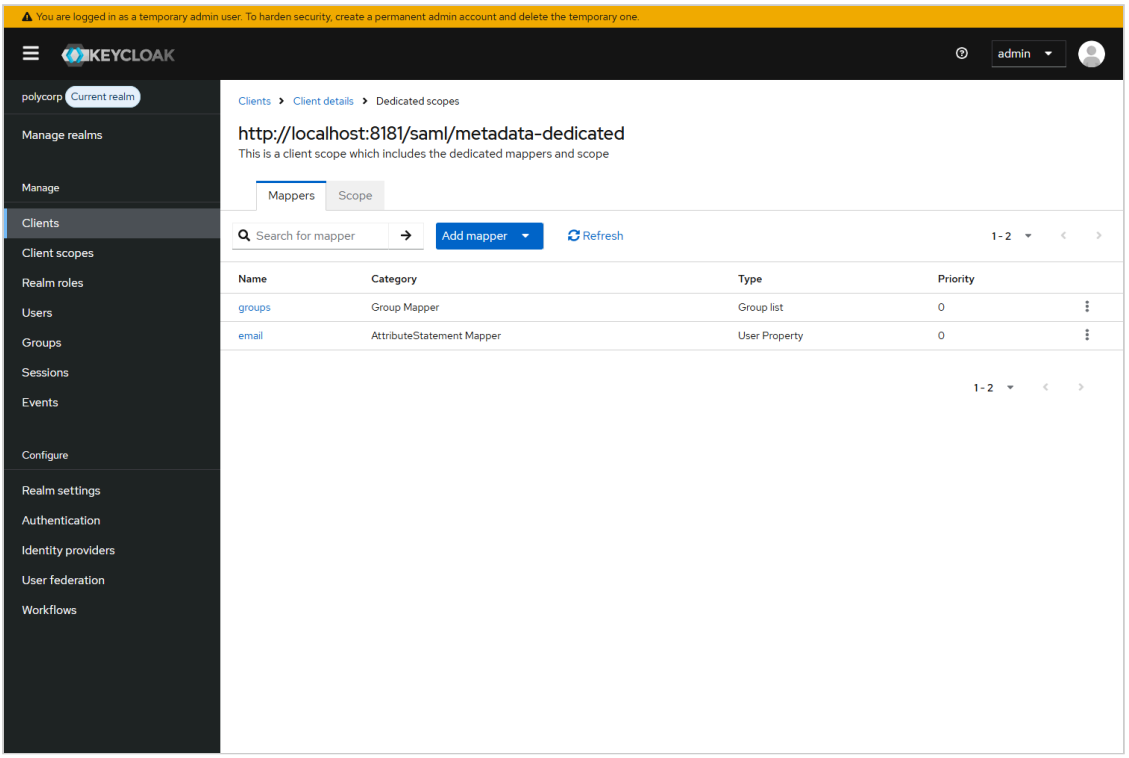


Figure 9 – *Mappers du client SAML : groups (Group list) et email (User Property)*

Côté OIDC

Le client `portail-notes` est **public** (pas de secret dans une application que l'utilisateur pourrait inspecter), avec le *Standard flow* et **PKCE S256 obligatoire**. Deux scopes optionnels décrivent ce que le client peut demander : `notes.read` et `notes.write`. Le mapper `aud-api-notes` ajoute `api-notes` dans l'audience de l'access token ; sans lui, l'API rejette le jeton. Le mapper `groups` publie les groupes dans l'ID token et l'access token. Enfin, le mapper *realm roles* écrit aussi les rôles dans l'ID token.

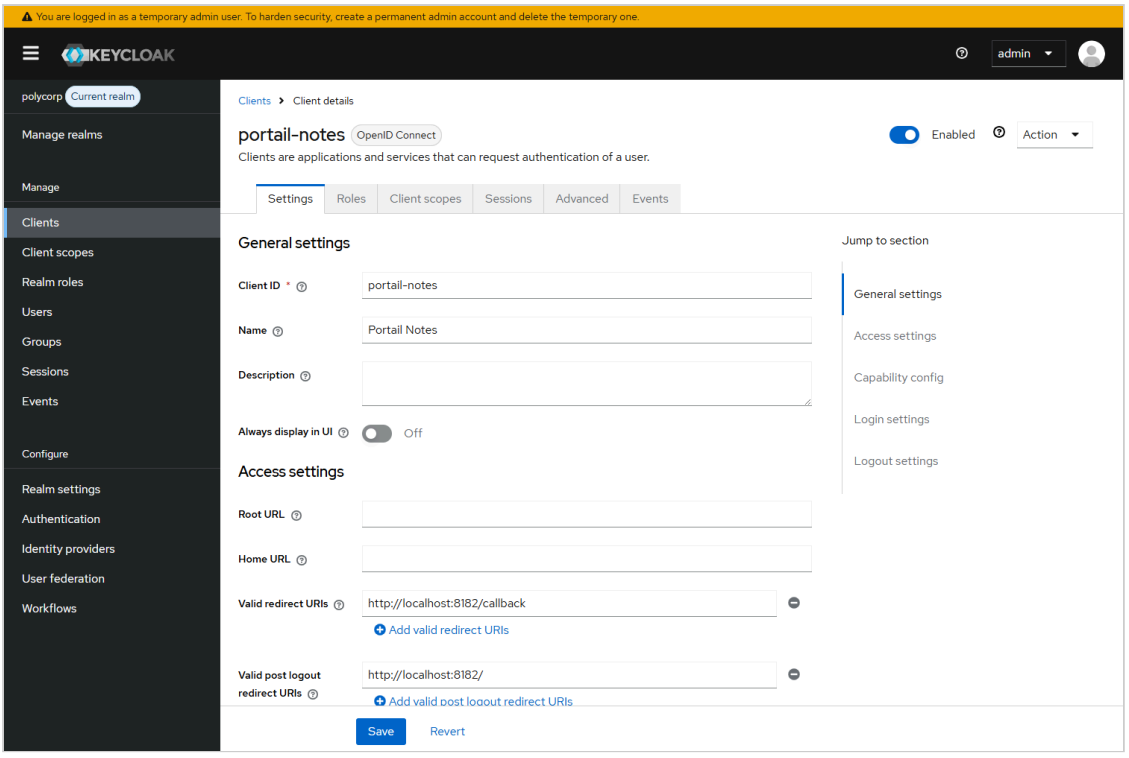


Figure 10 – *Client portail-notes : client public, redirect URI /callback, Standard flow*

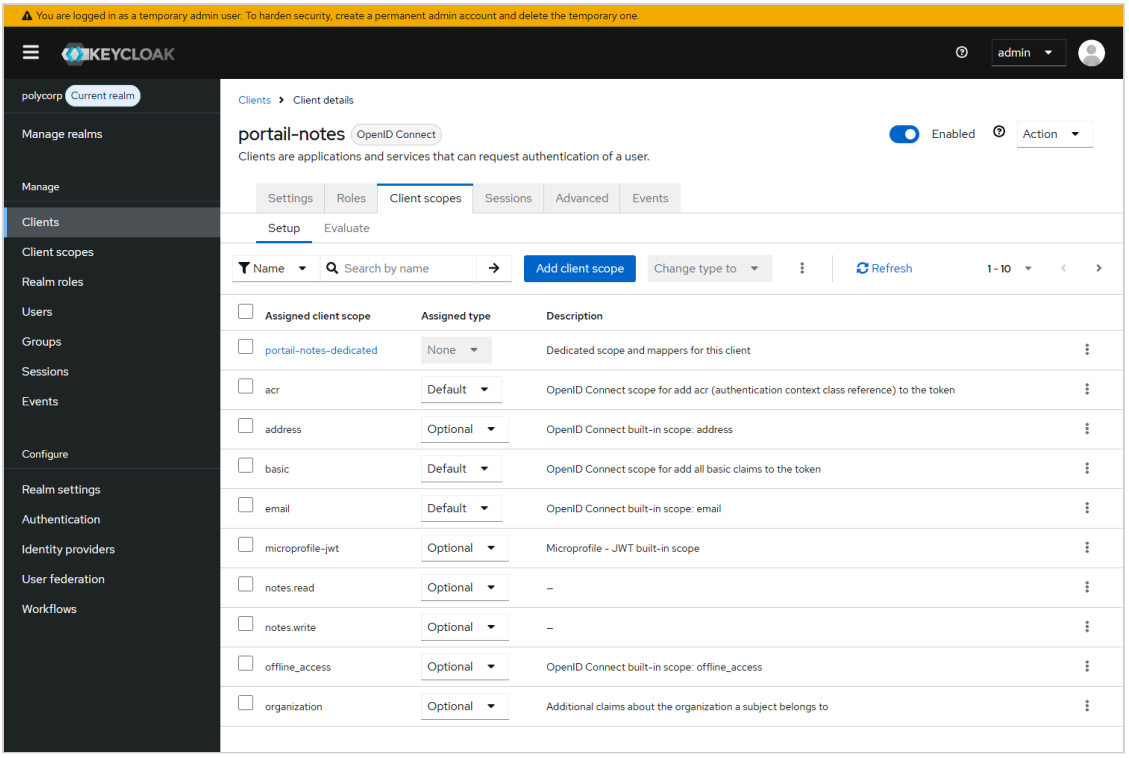


Figure 11 – *Scopes du client : notes.read et notes.write en Optional*

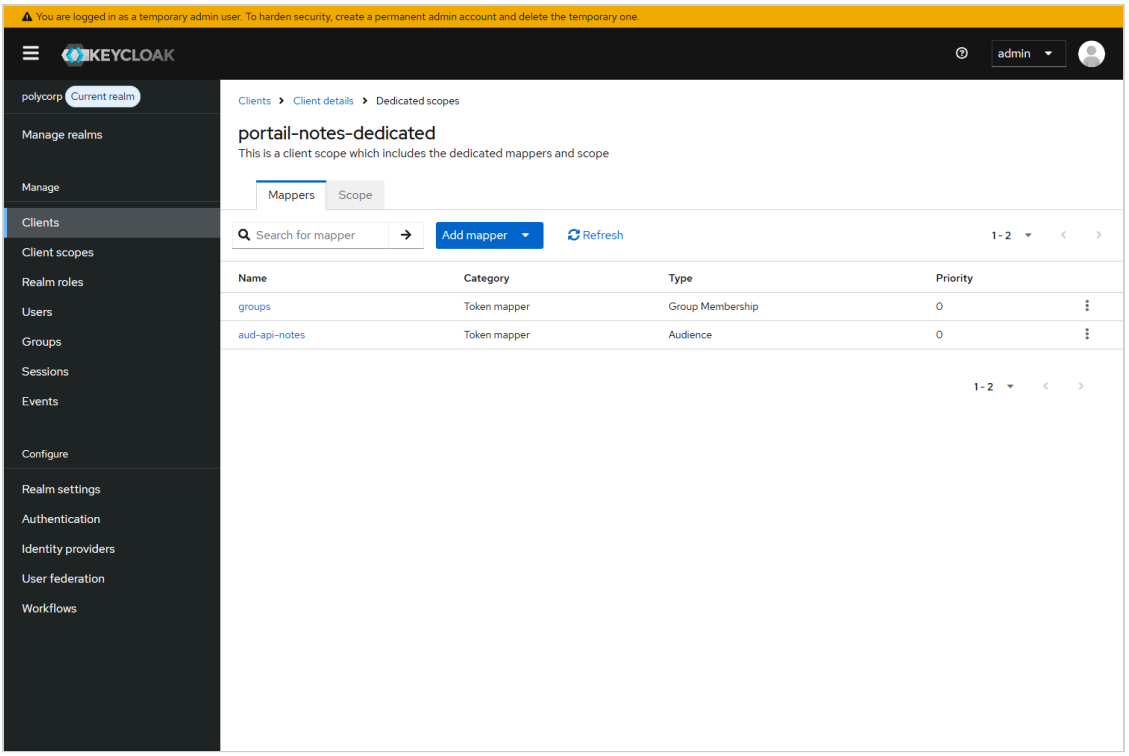


Figure 12 – *Mappers dédiés : audience api-notes et groups*

6. Les deux portails et le contrôle d'accès RH

6.1 Portail RH : fournisseur de service SAML

La confiance SAML est réciproque. Les métadonnées du SP (`sp-metadata.xml`) ont été importées dans Keycloak pour créer le client. Les métadonnées de l'IdP (`/realms/polycorp/protocol/saml/descriptor`) ont été chargées dans le SP, qui retient l'entityID, l'URL SSO et le certificat de signature.

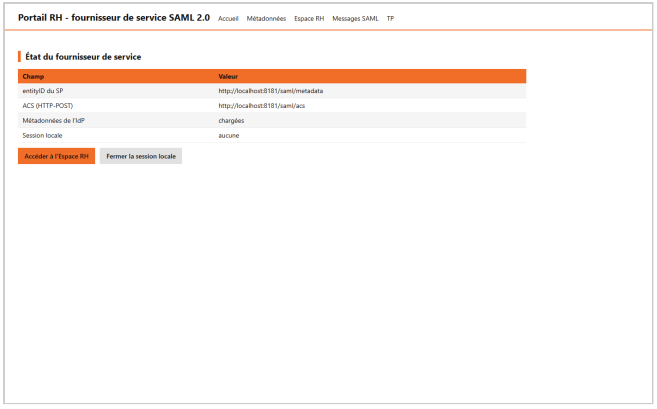


Figure 13 – *Accueil du SP : entityID, ACS, aucune session locale*

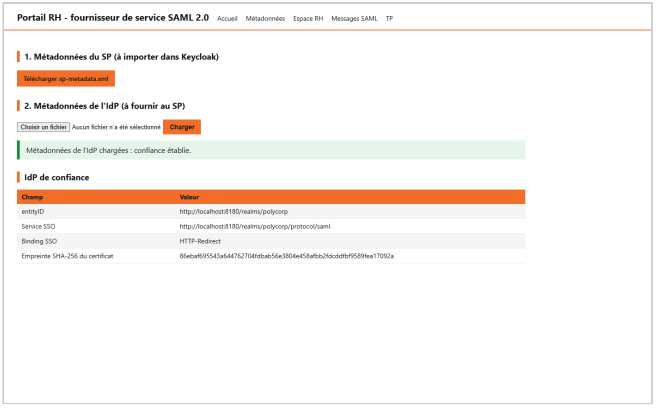


Figure 14 – *Métadonnées de l'IdP chargées : confiance établie*

Réservation de l'espace RH au groupe rh. Keycloak authentifie, mais c'est l'application qui autorise. Le SP lit les attributs `groups` et `Role` de l'assertion validée et exige la valeur `rh`. Cette valeur provient directement du groupe LDAP `cn=rh`.

TP4/apps/sp_saml.py (extrait)

```
GROUPE_RH = "rh"  # groupe LDAP requis pour l'Espace RH

@app.get("/rh")
def rh():
    if not session.get("nameid"):
        ...
        return redirect(url_for("connexion"))          # AuthnRequest vers Keycloak
    attrs = session.get("attributs", {})
    if GROUPE_RH not in set(attrs.get("groups", [])) | set(attrs.get("Role", [])):
        return vue(... "Accès refusé" ...), 403          # authentifié mais non autorisé
    return vue(... "accès accordé" ...)
```

Le SP vérifie aussi la signature de l'assertion (`wantAssertionsSigned`), la destination, l'audience, les dates, le `InResponseTo` (pas de réponse non sollicitée) et refuse de rejouer une assertion déjà consommée.

6.2 Portail Notes : client OIDC et API

Le portail montre chaque étape du flux *Authorization Code* : paramètres envoyés, `code_challenge` PKCE, code reçu, échange au point `/token`, puis validation de l'ID token (signature JWKS, `iss`, `aud`, `exp`, `nonce`).

L'API vérifie de son côté la signature, l'émetteur, l'audience `api-notes`, l'expiration, le scope et, pour l'écriture, le rôle `editeur`.

TP4/apps/api_notes.py (extrait)

```
@app.post("/notes")
def ecrire():
    c, controles, r = controler("notes.write", "editeur")   # scope du client + rôle de l'utilisateur
    if r:
        return r                                            # 401 jeton invalide / 403 scope ou rôle
```

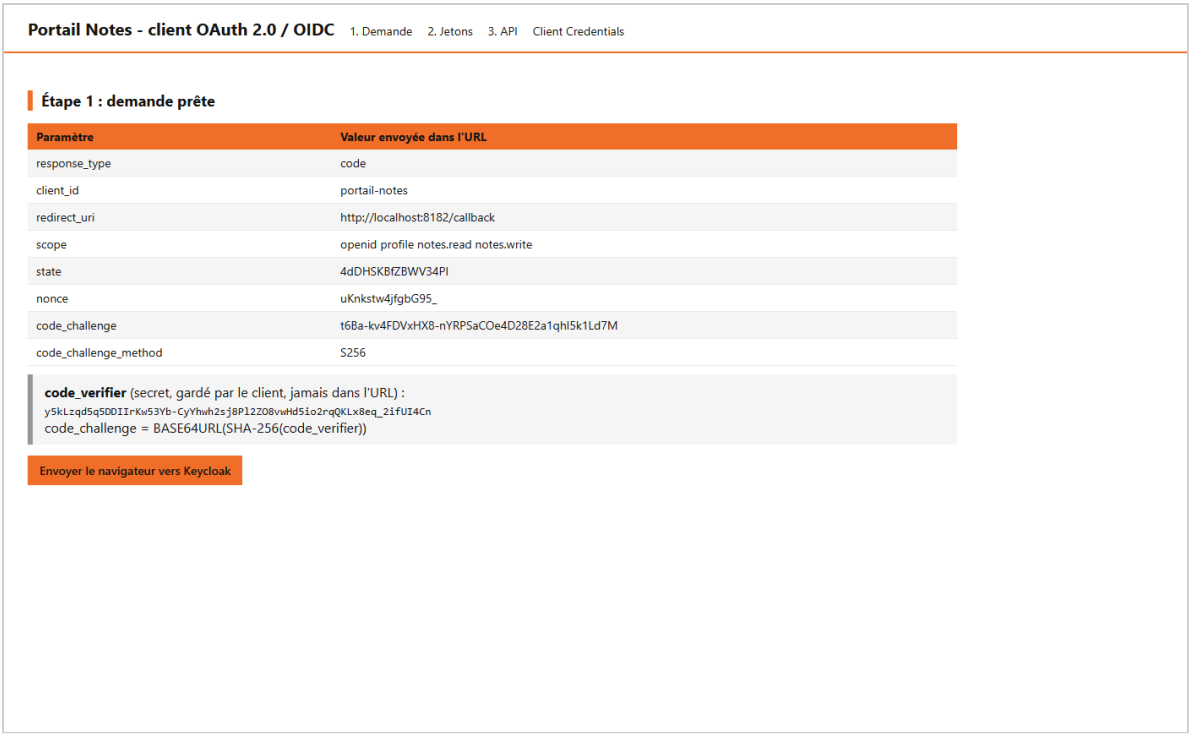


Figure 15 – *Demande d'autorisation préparée : scopes, state, nonce, code_challenge S256 ; le code_verifier reste côté client*

7. Démonstrations

7.1 Un seul mot de passe pour les deux portails

Alice ouvre `http://localhost:8181/rh`. N'ayant pas de session locale, le SP envoie une `AuthnRequest` à Keycloak, qui affiche le formulaire du realm **polycorp**. C'est la seule saisie du mot de passe : Keycloak le vérifie par un bind LDAP puis renvoie une assertion signée.

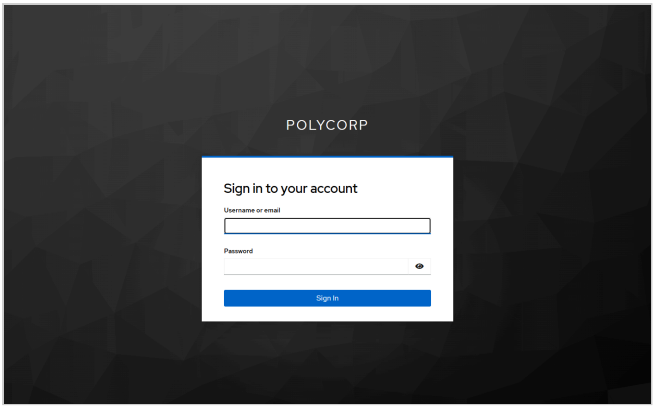


Figure 16 – *Formulaire du realm polycorp : unique saisie du mot de passe*

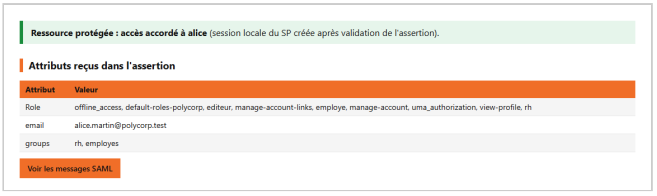


Figure 17 – *Portail RH : accès accordé à alice, attributs reçus*

Elle ouvre ensuite le Portail Notes et lance la demande d'autorisation. Keycloak reconnaît la session SSO (cookie du realm) et renvoie immédiatement un code, **sans afficher de formulaire**.

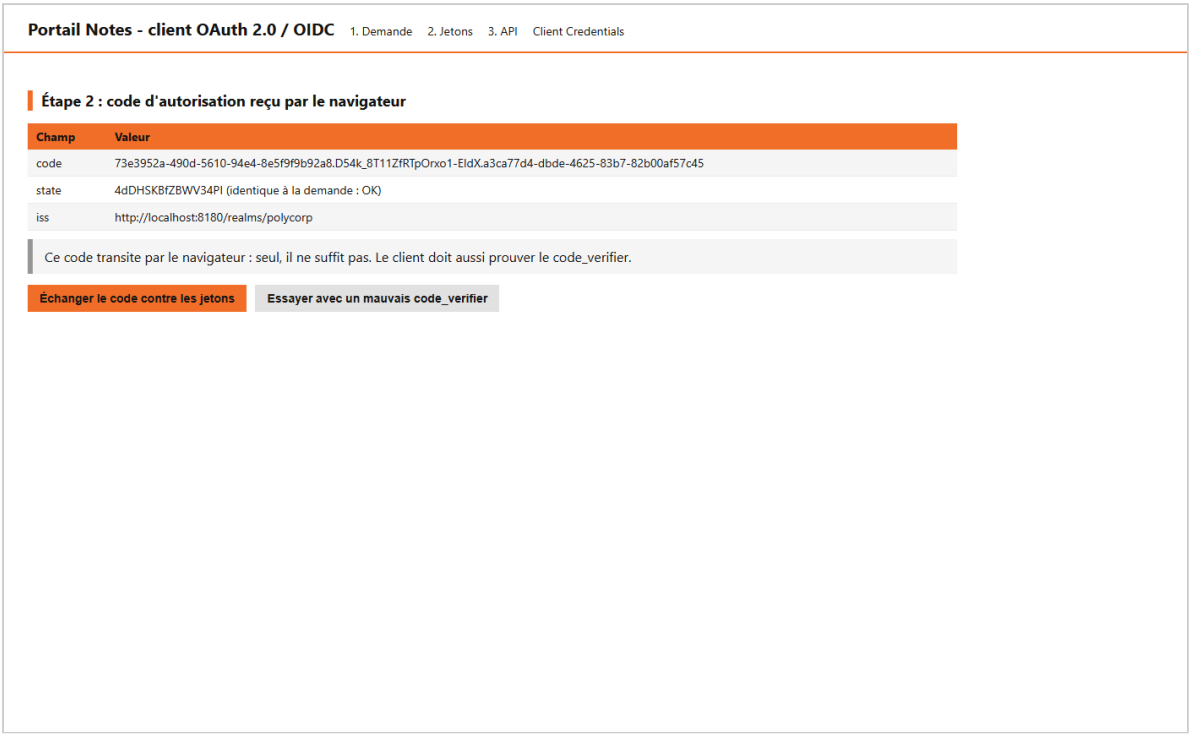


Figure 18 – *Portail Notes : code d'autorisation reçu directement, state vérifié, sans nouvelle saisie*

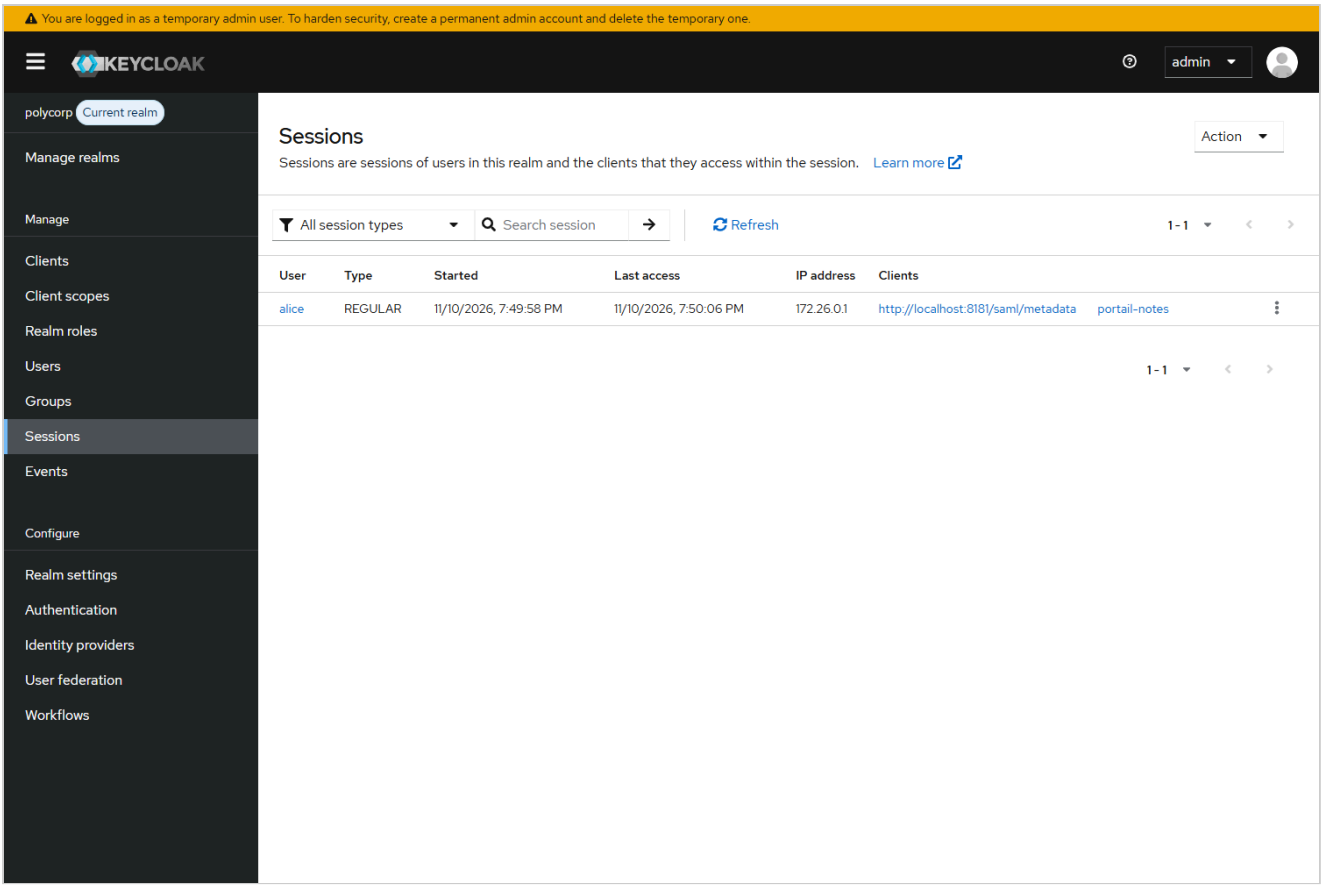


Figure 19 – *Console Keycloak, realm polycorp : une seule session pour alice, deux clients (SAML et portail-notes)*

Résultat : une authentification, une session IdP, deux applications et deux protocoles différents.

7.2 Le même utilisateur dans l'assertion SAML et dans l'ID token

Champ	Valeur
Response : Destination	http://localhost:8181/saml/acs
Response : InResponseTo	ONELOGIN_3d3bc50686d87f5fbde4b8563e4a1ef6e1e740b4
Response : Status	Success
Issuer (IdP)	http://localhost:8180/realms/polycorp
Assertion : ID	ID_686e35f6-34fd-401e-9ce3-ce447c32ddc5
Subject : NameID	alice
Subject : Recipient	http://localhost:8181/saml/acs
Conditions : NotBefore	2026-11-10T18:49:56.499Z
Conditions : NotOnOrAfter	2026-11-10T18:50:56.499Z
Conditions : Audience	http://localhost:8181/saml/metadata
AuthnStatement : AuthnInstant	2026-11-10T18:49:58.499Z
Signature de la Response	oui
Signature de l'Assertion	oui

Figure 20 – *Assertion SAML reçue par le Portail RH*


```
{
  "exp": 1794336906,
  "iat": 1794336606,
  "auth_time": 1794336598,
  "jti": "1e12f653-2f59-9389-2089-e230bbc4aa83",
  "iss": "http://localhost:8180/realms/polycorp",
  "aud": "portail-notes",
  "sub": "80dff7c0-1b22-4f67-a280-bc9283fad1e3",
  "typ": "ID",
  "azp": "portail-notes",
  "nonce": "uKnkstw4jfgb695_",
  "sid": "DS4k_8T11fRTpOrnol-EIdX",
  "at_hash": "GZzIppzCXWWRNmSKfHBZnQ",
  "acr": "0",
  "email_verified": false,
  "realm_access": {
    "roles": [
      "default-roles-polycorp",
      "employe",
      "rh",
      "offline_access",
      "uma_authorization",
      "editeur"
    ]
  },
  "name": "Alice Martin",
  "groups": [
    "/rh",
    "employes"
  ],
  "preferred_username": "alice",
  "given_name": "Alice",
  "family_name": "Martin",
  "email": "alice.martin@polycorp.test"
}
```


Figure 21 – *ID token décodé par le Portail Notes*

Information	Assertion SAML	ID token OIDC	Analyse
Sujet	`NameID = alice`	`sub = 89dff7c0-1b22-…-bc9283fad1e3`	Même personne. SAML transmet le login ; OIDC l'UUID Keycloak, stable même si le login change (le login est dans `preferred_username`)
Émetteur	`Issuer` = http://localhost:8180/realms/polycorp	`iss` = même valeur	Un seul IdP pour les deux protocoles
Destinataire	`Audience` = http://localhost:8181/saml/metadata	aud = portail-notes	Chaque preuve n'est valable que pour son destinataire
Fin de validité	`NotOnOrAfter` = NotBefore + 60 s	exp = iat + 300 s	Même rôle ; date ISO 8601 contre horodatage Unix
Droits	`Role` : rh, editeur, employe… ; `groups` : rh, employes	`realm_access.roles` ; `groups` : rh, employes	Mêmes droits, issus des groupes LDAP
Intégrité	Signature XML de la Response et de l'Assertion	Signature JWS RS256 (clé publique JWKS)	Le destinataire vérifie sans contacter l'IdP
Anti-rejeu	`InResponseTo` + ID d'assertion consommé une fois	nonce + code à usage unique	Lien entre la demande et la réponse

Contrôle	Résultat
signature (JWKS)	OK
iss	OK : http://localhost:8180/realms/polycorp
aud = client_id	OK : portail-notes
exp	OK : reste 299 s
nonce	OK

Figure 22 – *Validation de l'ID token par le client : signature, iss, aud, exp et nonce OK*

```
<samlp:AuthnRequest xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol" xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion" ID="ONELOGIN_3d3bc50686d87f5fbde4b8563e4a1ef6e1e740b4" Version="2.0" IssueInstant="2026-11-10T18:49:57Z" Destination="http://localhost:8180/realms/polycorp/protocol/saml" ProtocolBinding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST" AssertionConsumerServiceURL="http://localhost:8181/saml/acs">
    <saml:Issuer>http://localhost:8181/saml/metadata</saml:Issuer>
    <samlp:NameIDPolicy Format="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified" AllowCreate="true"/>

</samlp:AuthnRequest>
```

Figure 23 – *AuthnRequest envoyée par le SP (HTTP-Redirect)*

7.3 Emma est authentifiée, mais refusée

Après fermeture de la session locale du SP et de la session Keycloak, emma se connecte sur `/rh`. Son authentification réussit, mais l'assertion ne contient ni le groupe ni le rôle `rh` : le portail répond **403** et affiche les attributs reçus.

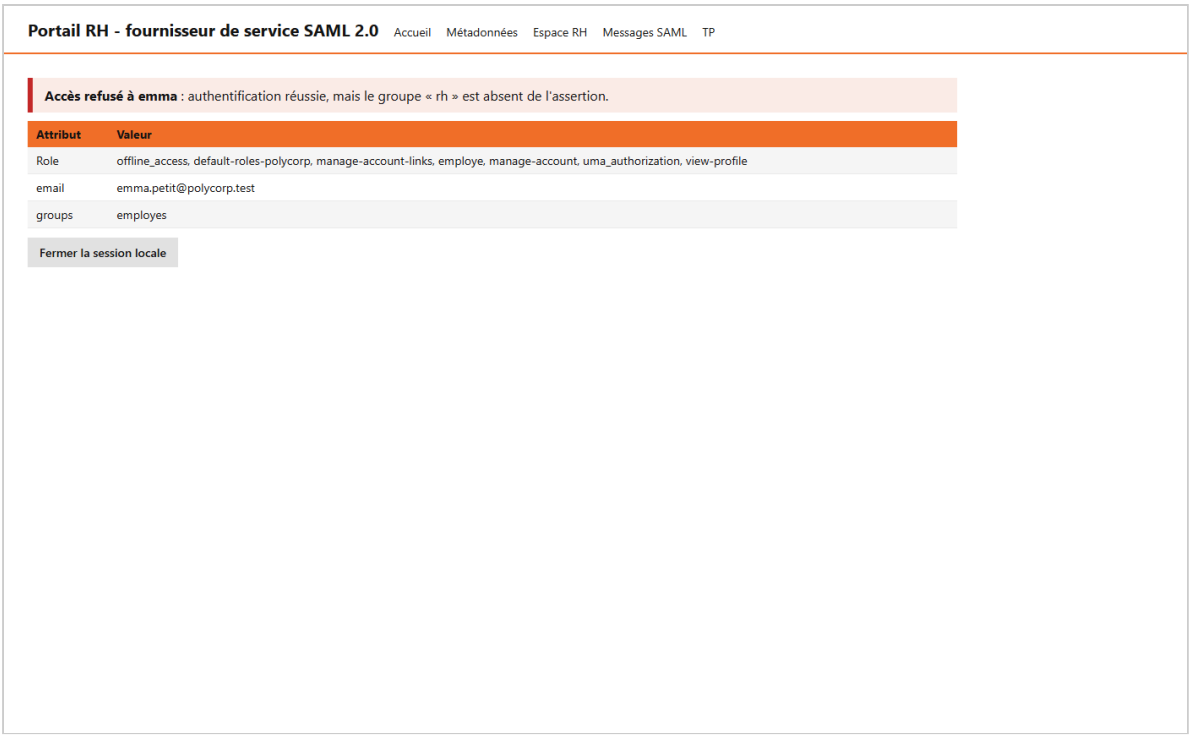


Figure 24 – *Portail RH : emma authentifiée, groups = employes, accès refusé (403)*

Sur le Portail Notes, emma obtient bien un jeton avec `notes.read` et `notes.write`. La lecture passe, mais l'écriture est refusée car elle n'a pas le rôle `editeur`.

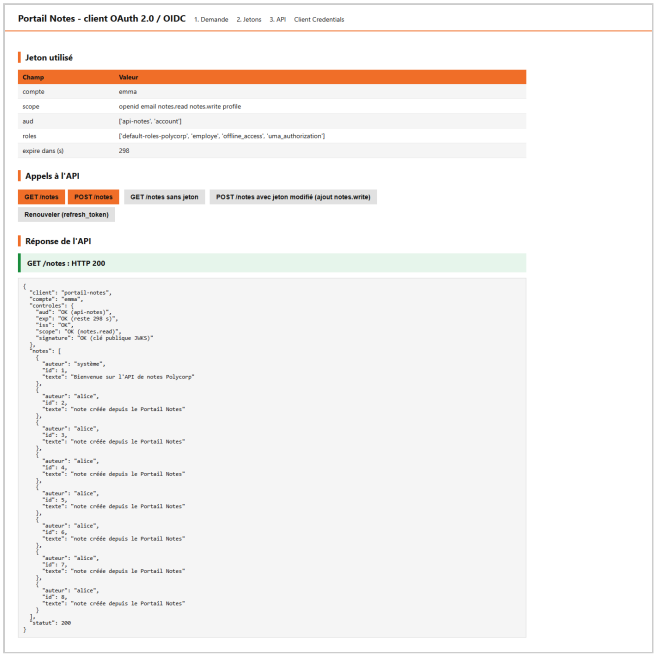


Figure 25 – *GET /notes : 200 (scope notes.read)*

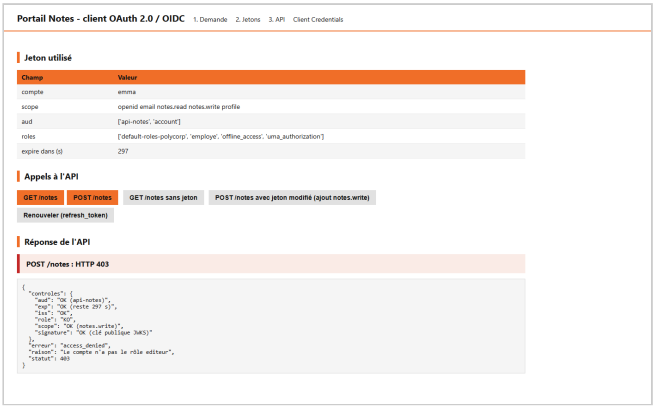


Figure 26 – *POST /notes : 403, rôle editeur absent*

> **À retenir :** le **scope** est ce que le client a le droit de demander, le **rôle** est ce que l'utilisateur a le droit de faire. L'API vérifie les deux. Par comparaison, alice obtient 201 sur le même appel (figure suivante).

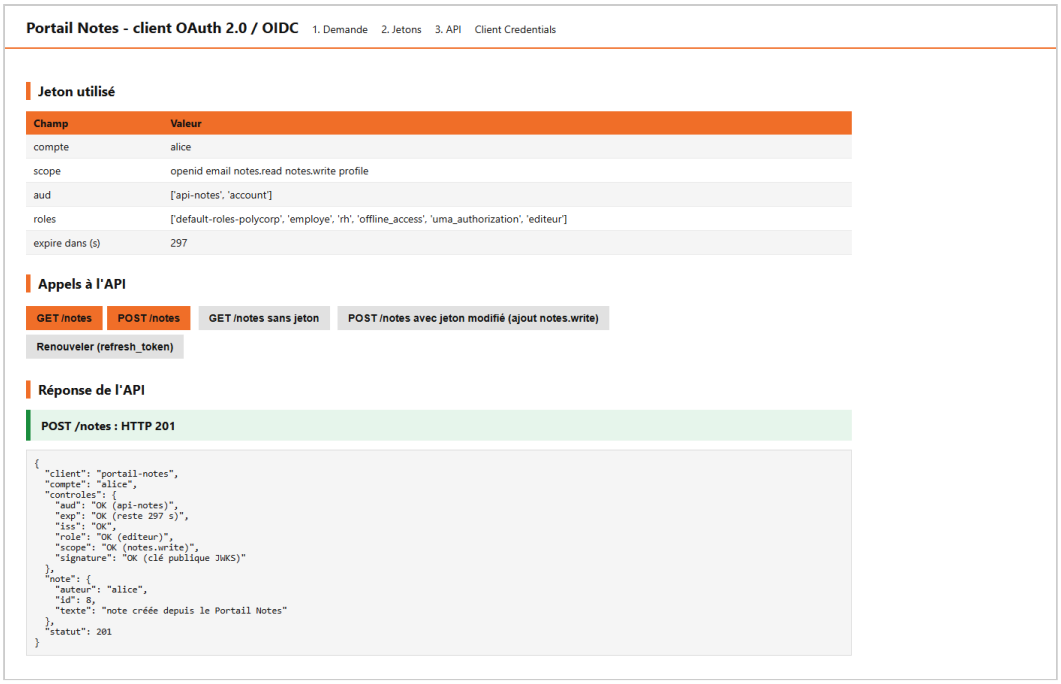


Figure 27 – *POST /notes avec le jeton d'alice : 201, rôle editeur présent*

> **Observation pendant les essais.** Un premier essai avait donné 201 pour « emma » : le portail utilisait encore le jeton d'alice gardé en mémoire. L'API ne regarde pas qui est devant le navigateur. Elle fait confiance au **jeton présenté**, valable jusqu'à exp.

7.4 Propagation d'un changement dans LDAP : mesure

Pour mesurer sans navigateur, nous avons créé temporairement un client public `probe` avec *Direct access grants*. Le script de mesure (équivalent PowerShell de `scripts/mesurer.sh`, car `jq` et `bc` sont absents sous Windows) change le mot de passe d'alice avec `ldappasswd` et ajoute emma au groupe `rh` avec `ldapmodify`. Il demande ensuite un jeton en boucle jusqu'à ce que le changement soit visible, puis il remet l'annuaire dans son état initial.

Changement dans LDAP	NO_CACHE	MAX_LIFESPAN = 1 min
Nouveau mot de passe accepté	**0,32 s**	**0,26 s**
Ancien mot de passe encore accepté ?	non	non
Rôle rh visible pour emma	**0,28 s**	**51,97 s**

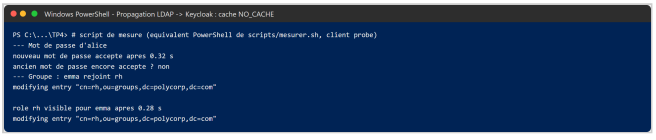


Figure 28 – *Mesure avec la politique NO_CACHE*

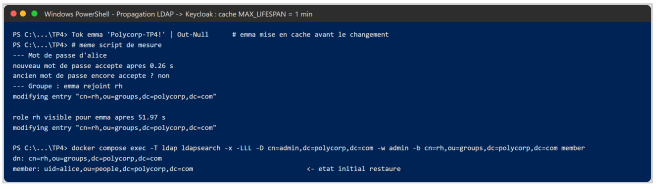


Figure 29 – *Mesure avec MAX_LIFESPAN = 1 minute*

Interprétation. Le mot de passe n'est jamais mis en cache : Keycloak le vérifie par un bind LDAP à chaque connexion, d'où une prise en compte immédiate dans les deux cas. Les groupes, eux, font partie de la

fiche utilisateur mise en cache. Avec `MAX_LIFESPAN`, Keycloak sert la copie en cache d'emma jusqu'à son expiration : les 51,97 s correspondent au temps restant sur cette minute, emma ayant été lue environ 8 s avant la modification.

> **Conséquence sur la révocation.** Le même délai s'applique quand on *retire* quelqu'un d'un groupe. Il faut y ajouter la durée de vie des preuves déjà émises : une assertion SAML (60 s ici) et surtout un access token (exp = 300 s), que l'API accepte jusqu'à expiration. Avec le cache d'une minute, un droit retiré dans LDAP peut donc survivre jusqu'à environ six minutes. Le cache réduit la charge sur l'annuaire au prix de la réactivité. Nous avons remis `NO_CACHE` après la mesure et supprimé le client `probe`.

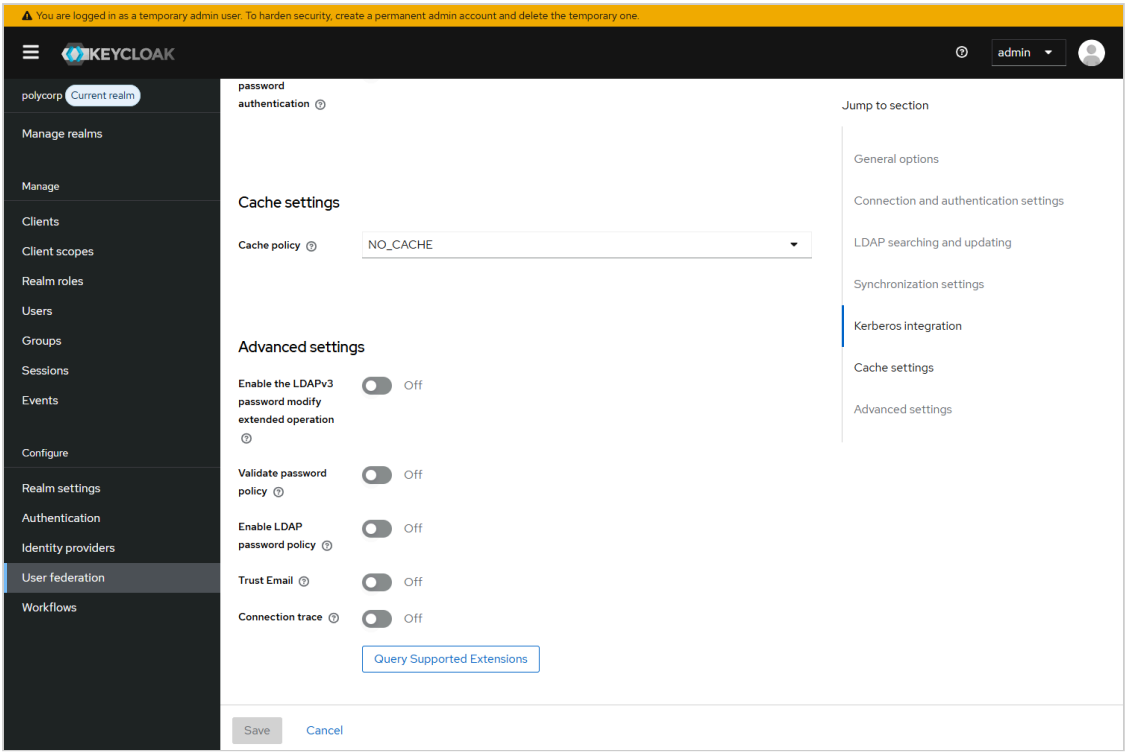


Figure 30 – *Politique de cache de la fédération LDAP (remise à NO_CACHE après la mesure)*

7.5 Le mot de passe n'est stocké que dans l'annuaire

La fiche d'alice dans Keycloak porte le **Federation link : ldap**. Ses champs (username, email, nom) sont en lecture seule, car ils viennent de l'annuaire. L'onglet *Credentials* n'affiche qu'une ligne « password — **Provided by ldap** ». Il s'agit d'une credential *virtuelle* : l'API d'administration la renvoie sans identifiant, avec une date de création à `-1` et le lien de fédération. Keycloak ne détient aucun hash.

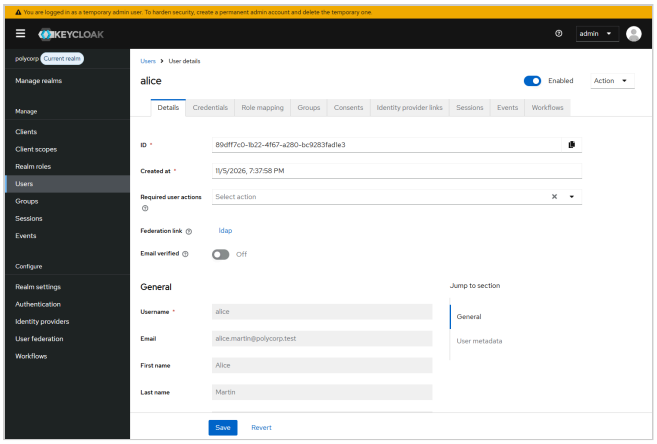


Figure 31 – *Fiche d'alice : Federation link ldap*

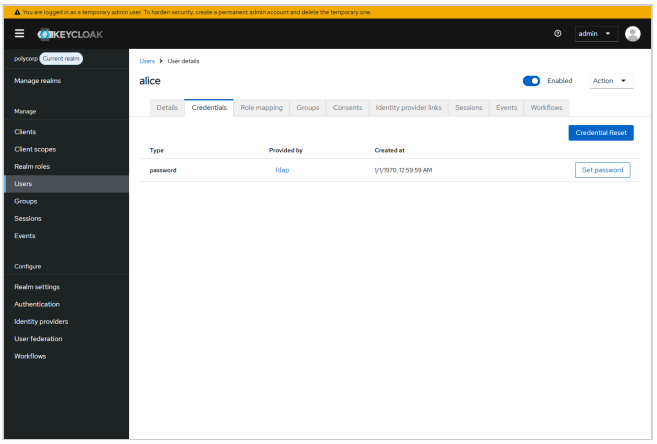


Figure 32 – *Credentials : mot de passe fourni par LDAP, non stocké*

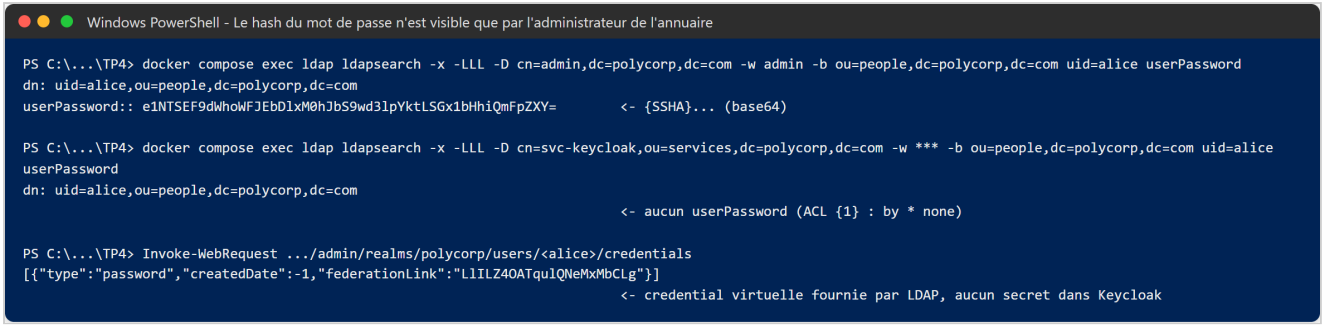


Figure 33 – *Le hash {SSHA} n'est lisible que par l'administrateur LDAP ; le compte de service ne le voit pas*

8. Kerberos : intégration possible avec la fédération LDAP

La fédération LDAP de Keycloak comporte une section **Kerberos integration** et crée par défaut un *Kerberos principal attribute mapper* (attribut `krb5PrincipalName`). Kerberos permettrait un SSO « de bureau » : un poste déjà connecté au domaine présente son ticket via SPNEGO (`Authorization: Negotiate`), et Keycloak ouvre la session sans aucun formulaire.

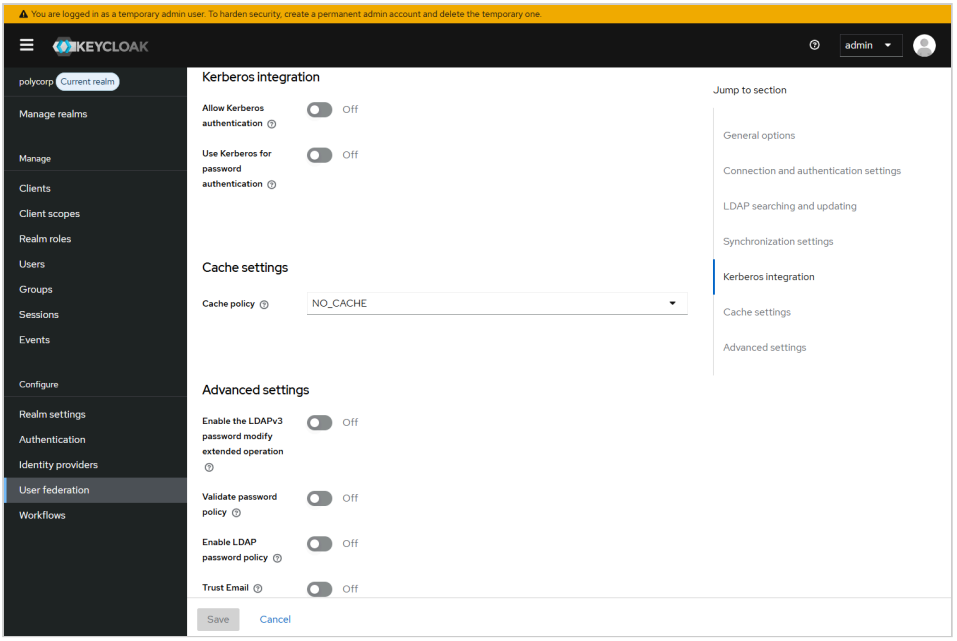


Figure 34 – *Section Kerberos de la fédération : Allow Kerberos authentication et Use Kerberos for password authentication désactivés*

Option	Valeur	Effet si activée
Allow Kerberos authentication	Off	Authentification SPNEGO avec le ticket du poste (nécessite un KDC, un keytab HTTP/<hôte> et la configuration du navigateur)
Use Kerberos for password authentication	Off	Le mot de passe saisi serait vérifié auprès du KDC au lieu d'un bind LDAP
Kerberos principal attribute	`krb5PrincipalName`	Lien entre le principal Kerberos et l'entrée LDAP de l'utilisateur

Pourquoi nous ne l'avons pas activé. Le laboratoire ne contient ni KDC (Active Directory ou MIT Kerberos), ni domaine pour les postes. Le sujet demandait une authentification adossée à l'annuaire : le bind LDAP suffit et reste la seule source de vérité pour le mot de passe. Dans un SI réel avec Active Directory, l'activer serait l'étape suivante logique : les utilisateurs déjà connectés à Windows n'auraient même plus à saisir leur mot de passe une fois.

9. Sécurité, limites et recommandations

9.1 Mécanismes de sécurité vérifiés

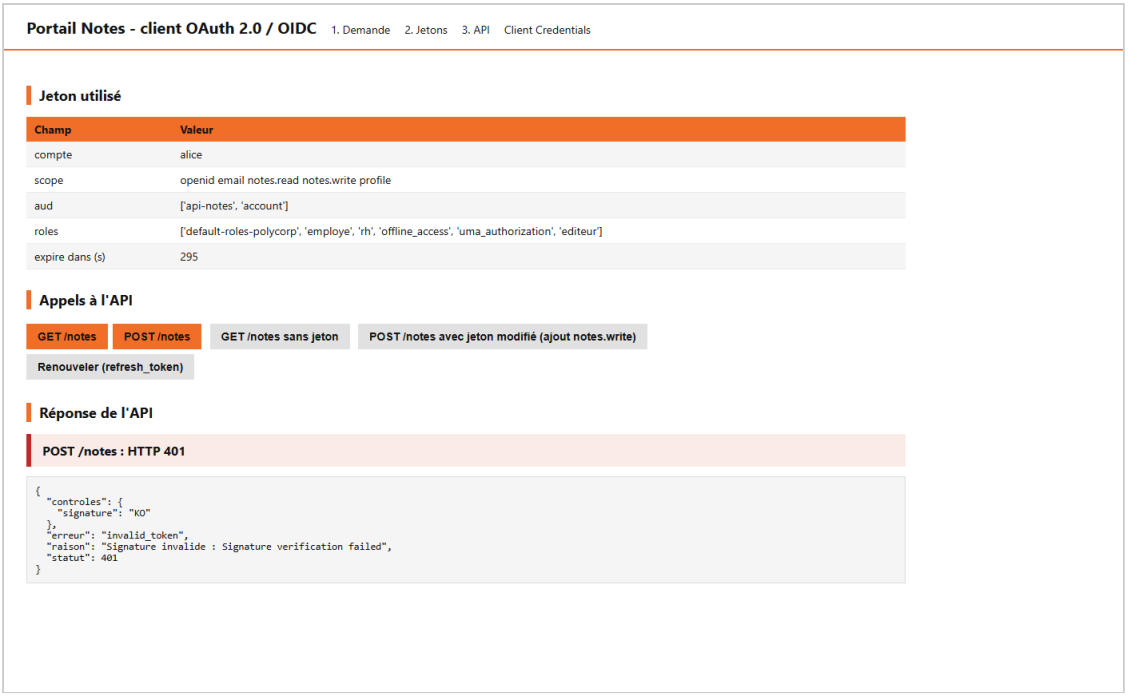


Figure 35 – *Jeton modifié (ajout de notes.write dans la charge utile sans re-signature) : rejeté en 401, signature invalide*

Menace	Contre-mesure en place
Modification d'un jeton par le client	Signature RS256 vérifiée par l'API avec les clés JWKS (figure ci-dessus)
Interception du code d'autorisation	PKCE S256 obligatoire, code à usage unique, `state` contre le CSRF
Jeton utilisé auprès d'un autre service	Contrôle de l'audience : `aud = api-notes` pour l'API, `Audience` SAML pour le SP
Rejeu d'une assertion SAML	`InResponseTo`, durée de 60 s, ID d'assertion consommé une seule fois
Fuite du compte de service	Lecture seule, sans accès à `userPassword` ni à `ou=services`
Vol des mots de passe dans l'IdP	Aucun mot de passe stocké dans Keycloak, hash SSHA uniquement dans LDAP

9.2 Limites du laboratoire et recommandations pour la production

Limite actuelle	Recommandation
`ldap://` en clair : le mot de passe transite en clair lors du bind	**LDAPS** (636) ou **StartTLS**, avec la vérification du certificat
Keycloak en `start-dev`, HTTP, administrateur temporaire `admin/admin`	Mode `start` derrière TLS, compte administrateur permanent, MFA pour la console
Hash SSHA (SHA-1 salé)	Schéma plus robuste (PBKDF2, Argon2 via module OpenLDAP) et politique de mots de passe
Métadonnées de l'IdP gardées en mémoire par le SP	Les persister (fichier ou URL de métadonnées rafraîchie), sinon un redémarrage casse la confiance
Révocation retardée (cache + durée de vie des jetons)	Access tokens courts, *refresh tokens* révocables, cache adapté au besoin, déconnexion globale
Contrôle `rh` codé dans l'application	Acceptable, mais à documenter ; on peut aussi l'imposer dans Keycloak (politique d'accès au client)
Client `probe` avec *password grant*	Réservé aux mesures, supprimé après usage ; ce flux est déconseillé (OAuth 2.1)

10. Conclusion

Nous avons ouvert deux applications de Polycorp sur l'annuaire existant sans recréer un seul compte. OpenLDAP reste la seule source des identités, des groupes et des mots de passe. Keycloak les fédère en lecture seule avec un compte de service limité par des ACL, puis les traduit en assertions SAML pour le Portail RH et en jetons OIDC pour le Portail Notes.

Les démonstrations confirment les objectifs : une seule saisie du mot de passe pour deux protocoles ; le même utilisateur reconnu dans l'assertion et dans l'ID token ; une autorisation fine (emma authentifiée mais refusée) fondée sur les groupes LDAP ; des changements visibles en moins d'une demi-seconde sans cache, et jusqu'à une minute avec cache ; enfin, l'absence de tout mot de passe dans Keycloak.

Le projet montre aussi que l'authentification et l'autorisation sont deux étapes distinctes. Keycloak dit *qui* est l'utilisateur ; chaque application et l'API décident *ce qu'il peut faire*, à partir des groupes, des rôles et des scopes. Il met enfin en évidence le compromis entre performance et réactivité de la révocation, qui doit guider le choix du cache et des durées de vie des jetons en production.

11. Annexe : procédure de reproduction

Commandes principales

```
# 1. Démarrer la plateforme (depuis le dossier TP4)
cd TP4
docker compose up -d --build

# 2. Appliquer puis tester les ACL de l'annuaire
docker compose cp ldap/02-acl.ldif ldap:/tmp/02-acl.ldif
docker compose exec ldap ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/02-acl.ldif
bash scripts/tester-acl.sh          # ou l'équivalent PowerShell

# 3. Keycloak (http://localhost:8180/admin, realm polycorp)
#    - User federation > LDAP : ldap://ldap:389, bind svc-keycloak, READ_ONLY, mapper de groupes
#    - Groupes rh -> rôles rh, editeur ; employes -> rôle employe
#    - Client SAML : import de sp-metadata.xml, mapper Group list "groups"
#    - Client portail-notes : public, PKCE S256, scopes notes.read / notes.write, mappers aud-api-notes
et groups

# 4. Portail RH : charger les métadonnées de l'IdP
#    http://localhost:8180/realms/polycorp/protocol/saml/descriptor  ->  http://localhost:8181/config

# 5. Démonstrations
#    http://localhost:8181/rh (alice puis emma)  /  http://localhost:8182 (Demande > Jetons > API)
#    Mesure : client public "probe" (Direct access grants), script de mesure, puis suppression

# 6. Le hash n'est visible que par l'administrateur LDAP
docker compose exec ldap ldapsearch -x -D cn=admin,dc=polycorp,dc=com -w admin -b ou=people,dc=polycor
p,dc=com uid=alice userPassword
```

Comptes de test

Compte	Groupes LDAP	Accès attendu
alice	rh, employes	Espace RH autorisé ; lecture et écriture des notes
bob	employes	Espace RH refusé ; lecture des notes
emma	employes	Espace RH refusé ; lecture des notes, écriture refusée

Les captures ont été produites automatiquement (Playwright + Microsoft Edge) sur la plateforme en fonctionnement le 10 novembre 2026 ; les sorties de terminal sont celles des exécutions réelles.